SYSTEM ANALYSIS & DESIGN — GAMESTORE PLATFORM

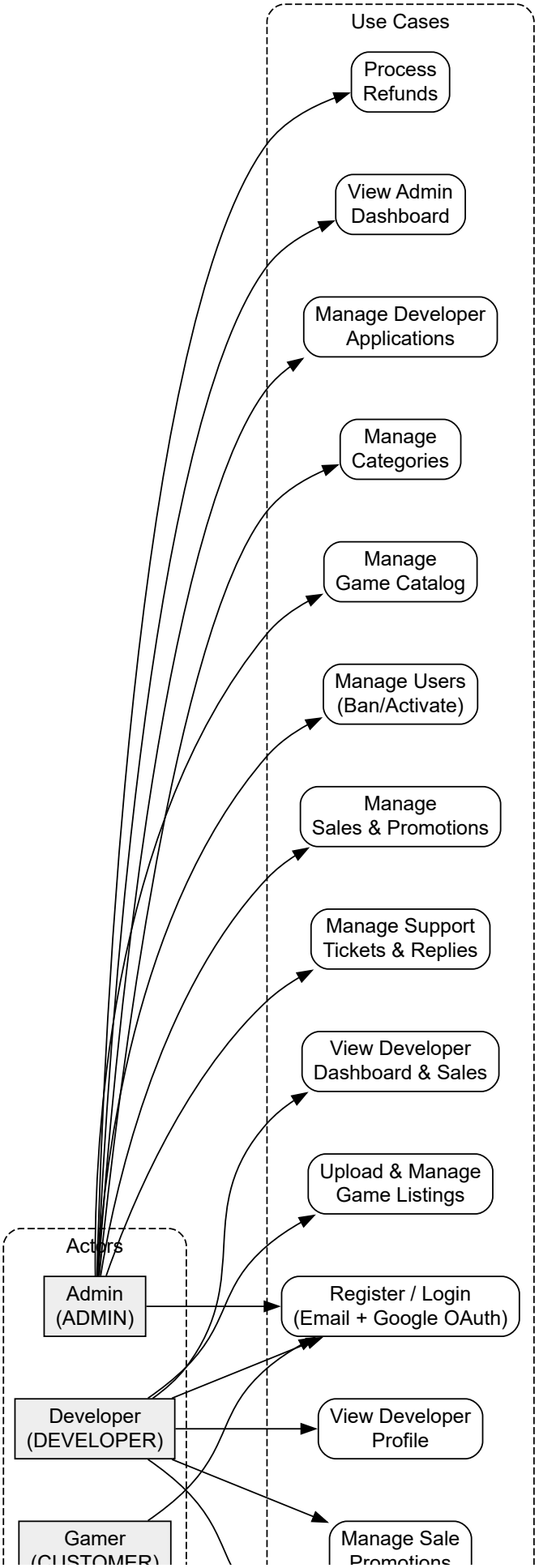
1. Problem Statement

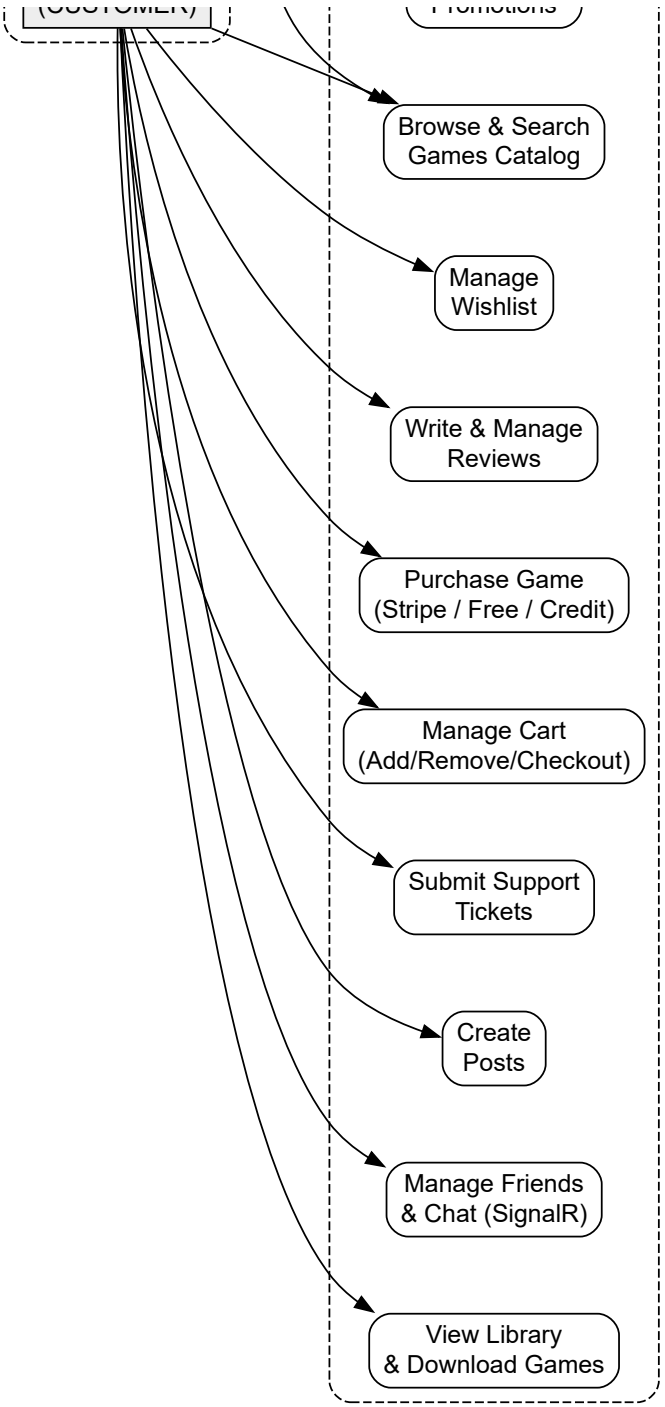
The GameStore platform aims to create a comprehensive digital marketplace for video games, connecting developers with gamers worldwide. The system must address the following objectives:

1. **1. User Management & Authentication:** Provide secure registration, login, and role-based access control supporting CUSTOMER, DEVELOPER, and ADMIN roles with cookie-based authentication (.GameStore.Auth), Google OAuth, session management, email verification, and password reset.
2. **2. Game Publishing & Catalog Management:** Enable developers to upload, update, and manage game listings with metadata (title, description, price, screenshots, trailer URL), categories (many-to-many via GameCategory), and game files, while administrators oversee catalog curation.
3. **3. Commerce & Payment Processing:** Implement a complete purchase workflow including cart management, Stripe Checkout Sessions integration, webhook-based payment confirmation (/stripe/webhook), store credit system, free checkout path, and commission-based payout calculations for developers.
4. **4. Community & Social Features:** Support user engagement through reviews/ratings (1-5 scale, one per user per game), wishlists, game libraries, real-time chat via SignalR (/hub/notifications), friendship system, and user posts.
5. **5. Content Moderation & Security:** Provide administrative tools for monitoring support tickets, reviewing game sales, managing developer applications, reviewing reported content, with rate limiting on auth endpoints, anti-forgery tokens, and security headers (X-Frame-Options DENY, X-Content-Type-Options nosniff).
6. **6. Analytics & Reporting:** Deliver dashboards for administrators and developers with sales trends, order analytics, user engagement metrics, popular categories, developer earnings, and platform growth indicators via dedicated dashboard controllers.

2. Use Case Diagram

The diagram below illustrates the three actor roles on the GameStore platform: Gamers (CUSTOMER), Developers (DEVELOPER), and Administrators (ADMIN), along with their associated use cases derived from the actual controller and service structure.

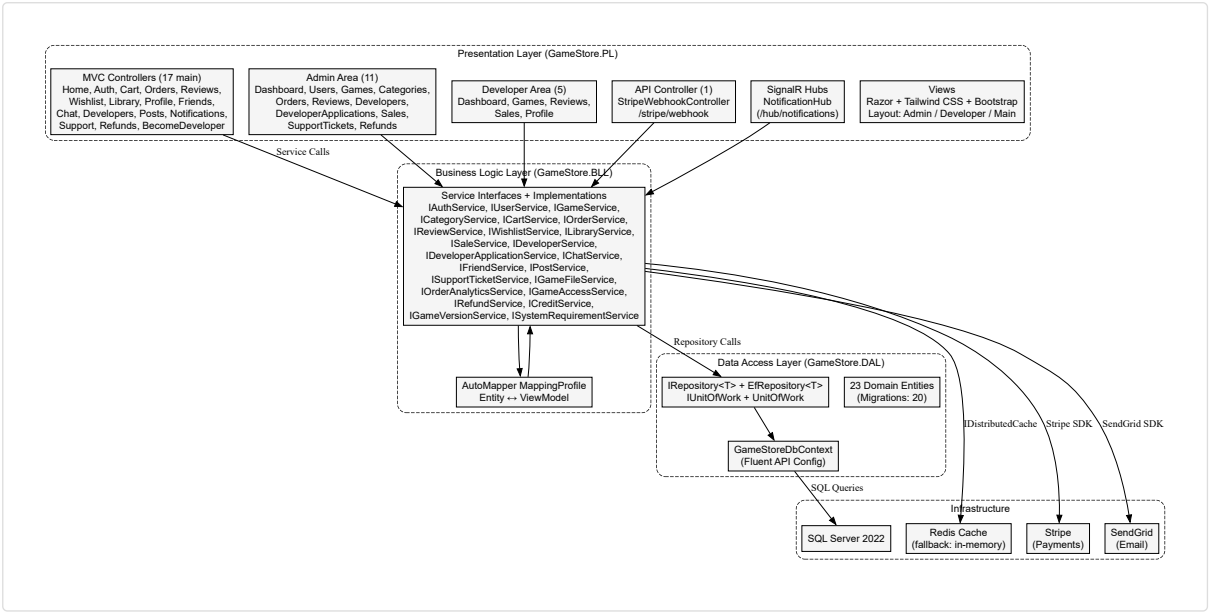

Use Cases
Process Refunds
View Admin Dashboard
Manage Developer Applications
Manage Categories
Manage Game Catalog
Manage Users (Ban/Activate)
Manage Sales & Promotions
Manage Support Tickets & Replies
View Developer Dashboard & Sales
Upload & Manage Game Listings
Register / Login (Email + Google OAuth)
View Developer Profile
Manage Sale Promotions
Actors
Admin (ADMIN)
Developer (DEVELOPER)
Gamer (CUSTOMER)



Actors map to the three Role enum values: CUSTOMER (0), DEVELOPER (1), ADMIN (2). Use cases derive from 33 MVC controllers across main, Admin area (11), and Developer area (5) areas.

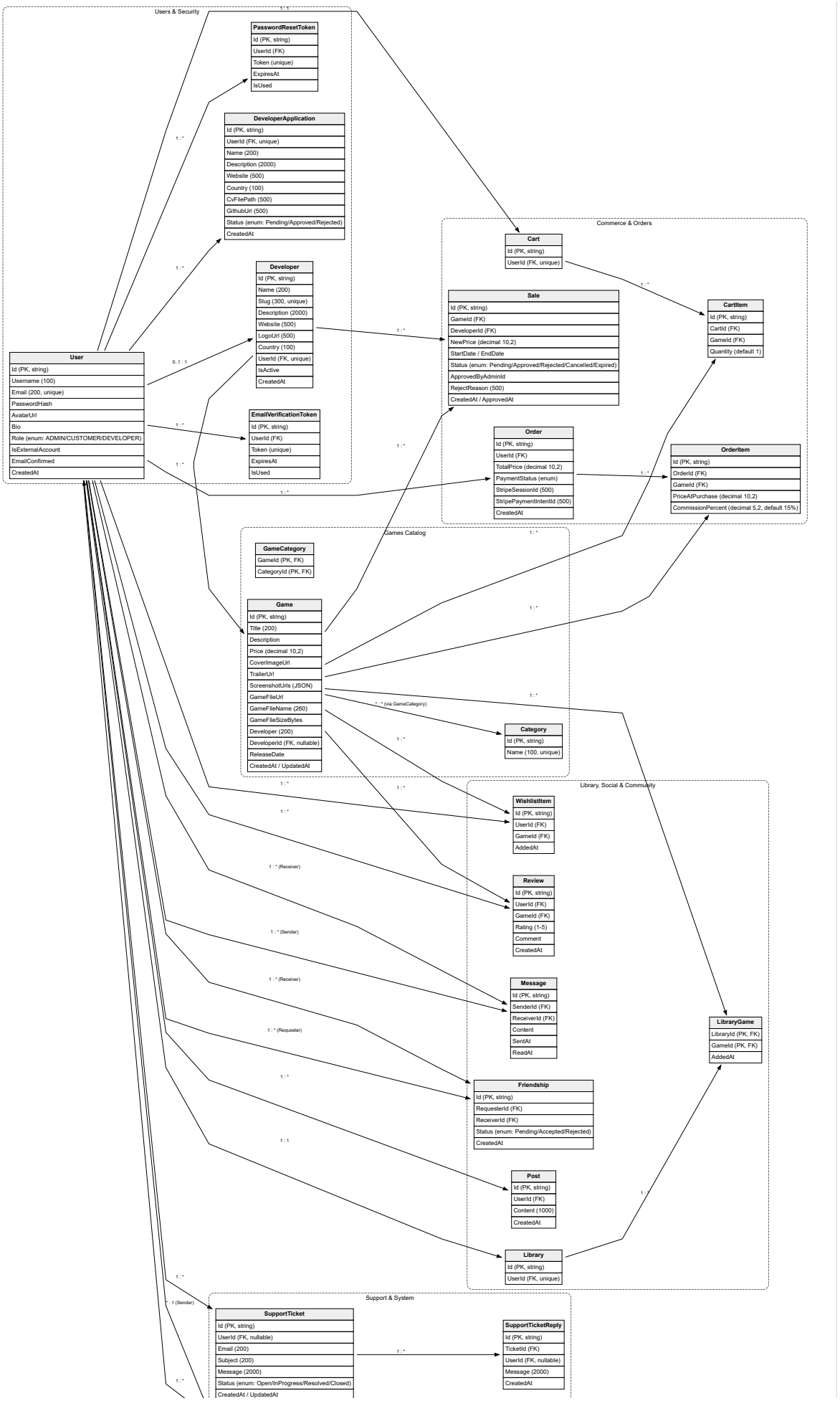
3. Three-Layer Architecture

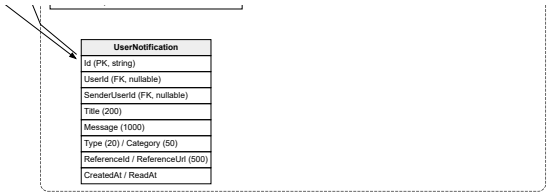
The system follows a three-layer architecture (PL → BLL → DAL) with additional infrastructure services for caching, real-time communication, and external integrations.



4. Entity Relationship Diagram

The ERD below maps all 23 domain entities with their actual relationships as configured in the GameStoreDbContext Fluent API.

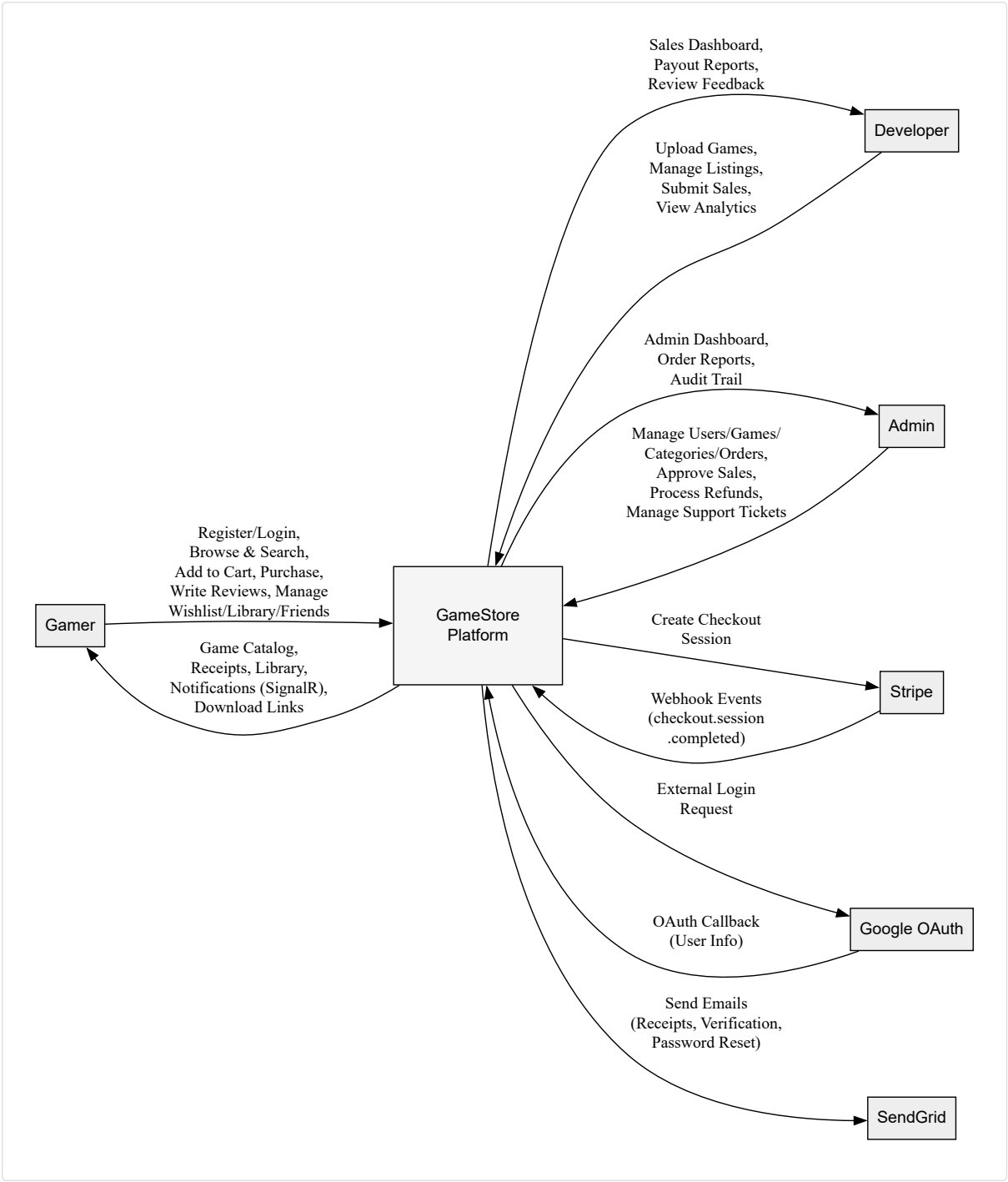

Users & Security
PasswordResetToken
Id (PK, string)
UserId (FK)
Token (unique)
ExpiresAt
IsUsed
DeveloperApplication
Id (PK, string)
UserId (FK, unique)
Name (200)
Description (2000)
Website (500)
Country (100)
CvFilePath (500)
GithubUrl (500)
Status (enum: Pending/Approved/Rejected)
CreatedAt
Developer
Id (PK, string)
Name (200)
Slug (300, unique)
Description (2000)
Website (500)
LogoUrl (500)
Country (100)
UserId (FK, unique)
IsActive
CreatedAt
User
Id (PK, string)
Username (100)
Email (200, unique)
PasswordHash
AvatarUrl
Bio
Role (enum: ADMIN/CUSTOMER/DEVELOPER)
IsExternalAccount
EmailConfirmed
CreatedAt
EmailVerificationToken
Id (PK, string)
UserId (FK)
Token (unique)
ExpiresAt
IsUsed
Commerce & Orders
Cart
Id (PK, string)
UserId (FK, unique)
Sale
Id (PK, string)
GameId (FK)
DeveloperId (FK)
NewPrice (decimal 10,2)
StartDate / EndDate
Status (enum: Pending/Approved/Rejected/Cancelled/Expired)
ApprovedByAdminId
RejectReason (500)
CreatedAt / ApprovedAt
CartItem
Id (PK, string)
CartId (FK)
GameId (FK)
Quantity (default 1)
Order
Id (PK, string)
UserId (FK)
TotalPrice (decimal 10,2)
PaymentStatus (enum)
StripeSessionId (500)
StripePaymentIntentId (500)
CreatedAt
OrderItem
Id (PK, string)
OrderId (FK)
GameId (FK)
PriceAtPurchase (decimal 10,2)
CommissionPercent (decimal 5,2, default 15%)
Games Catalog
GameCategory
GameId (PK, FK)
CategoryId (PK, FK)
Game
Id (PK, string)
Title (200)
Description
Price (decimal 10,2)
CoverImageUrl
TrailerUrl
ScreenshotUrls (JSON)
GameFileUrl
GameFileName (260)
GameFileSizeBytes
Developer (200)
DeveloperId (FK, nullable)
ReleaseDate
CreatedAt / UpdatedAt
Category
Id (PK, string)
Name (100, unique)
Library, Social & Community
WishlistItem
Id (PK, string)
UserId (FK)
GameId (FK)
AddedAt
Review
Id (PK, string)
UserId (FK)
GameId (FK)
Rating (1-5)
Comment
CreatedAt
Message
Id (PK, string)
SenderId (FK)
ReceiverId (FK)
Content
SentAt
ReadAt
LibraryGame
LibraryId (PK, FK)
GameId (PK, FK)
AddedAt
Friendship
Id (PK, string)
RequesterId (FK)
ReceiverId (FK)
Status (enum: Pending/Accepted/Rejected)
CreatedAt
Post
Id (PK, string)
UserId (FK)
Content (1000)
CreatedAt
Library
Id (PK, string)
UserId (FK, unique)
Support & System
SupportTicket
Id (PK, string)
UserId (FK, nullable)
Email (200)
Subject (200)
Message (2000)
Status (enum: Open/InProgress/Resolved/Closed)
CreatedAt / UpdatedAt
SupportTicketReply
Id (PK, string)
TicketId (FK)
UserId (FK, nullable)
Message (2000)
CreatedAt
1 : 1
1 : *
0..1 : 1
* : * (via GameCategory)
1 : * (Receiver)
1 : * (Sender)
1 : * (Requester)
1 : 1 (Sender)



All 23 entities use string GUIDs as primary keys. Relationships are configured via EF Core Fluent API with cascade/restrict delete behaviors as appropriate. Many-to-many relationships (Game↔Category) are modeled via the GameCategory join entity.

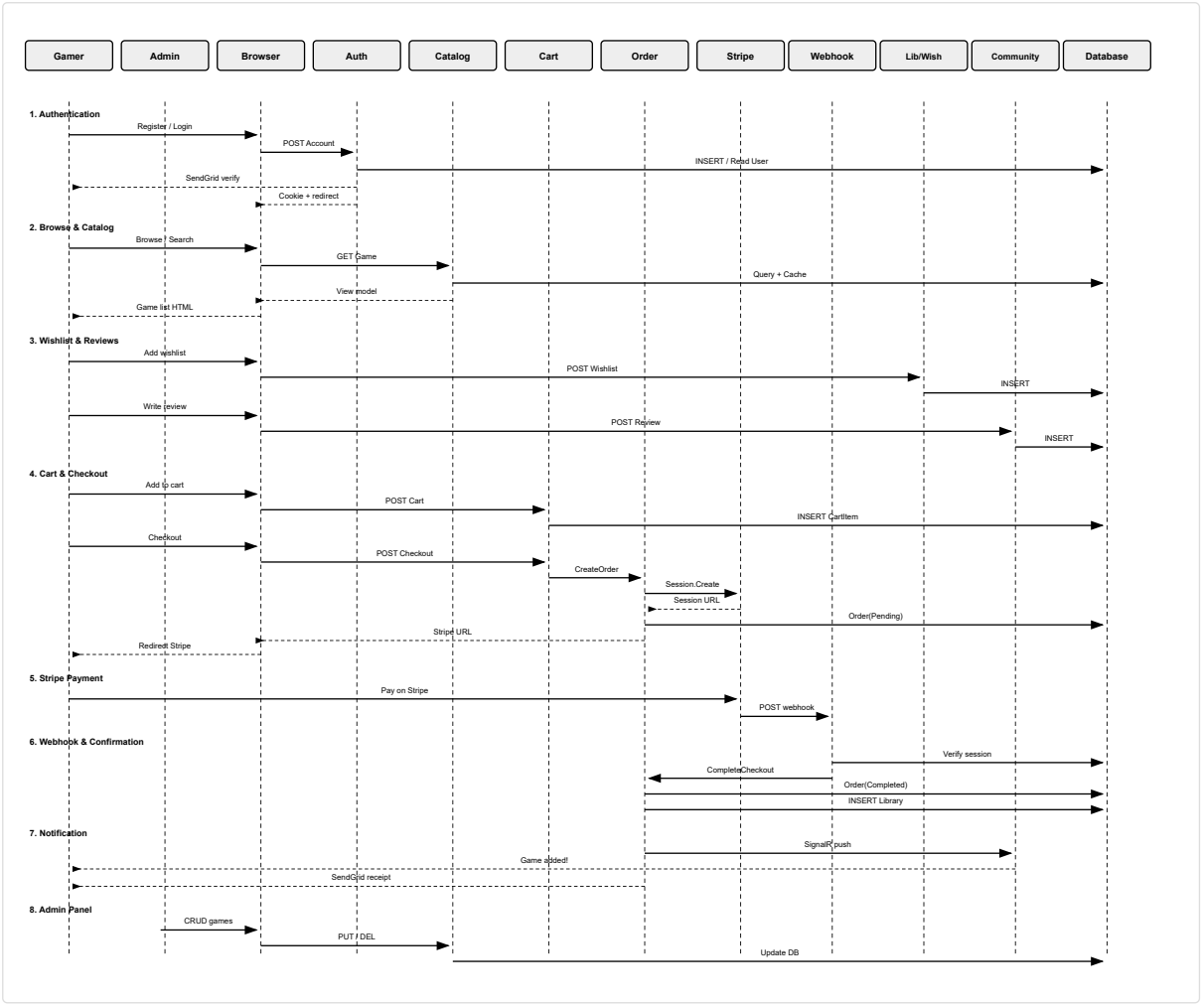
5. Context Diagram (DFD Level 0)

The context diagram shows the GameStore platform system boundary, external entities, and the primary data flows between them.



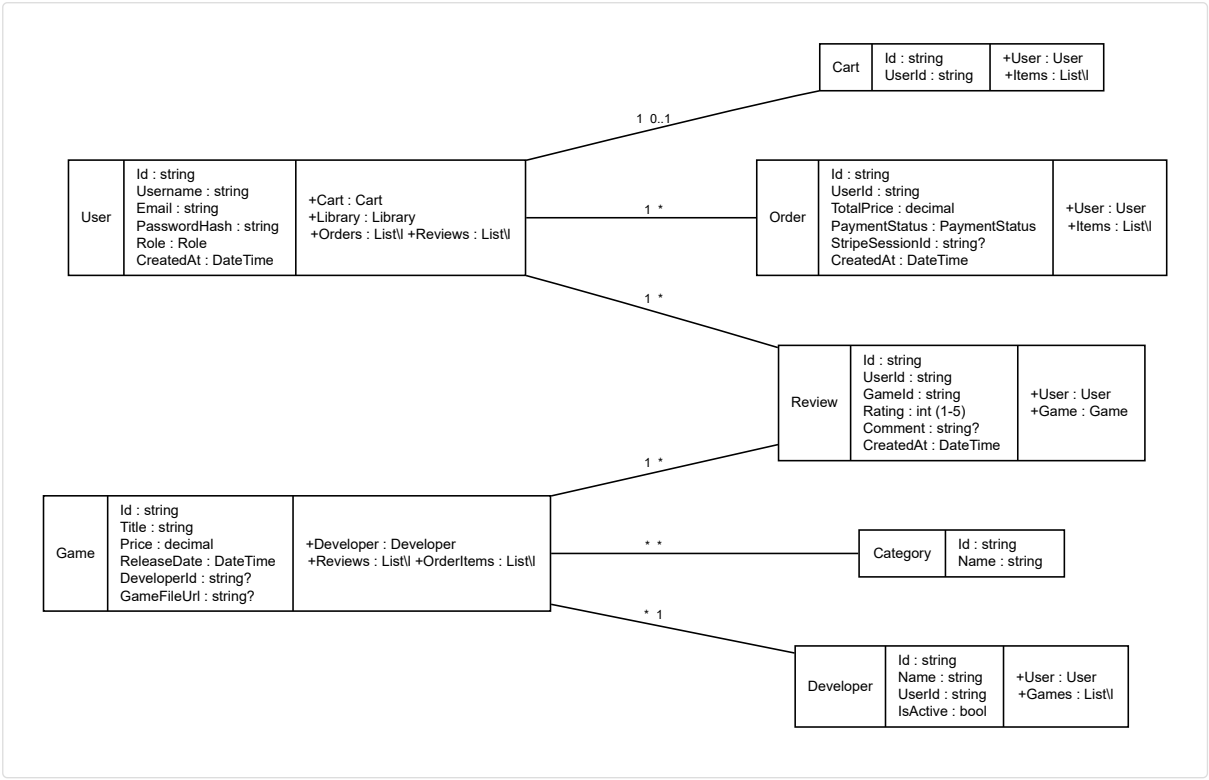
6. Purchase Sequence Diagram

This sequence diagram models the end-to-end purchase flow: browsing the catalog, adding a game to the cart, checking out via Stripe Checkout Session, webhook confirmation, and library access.



7. UML Class Diagram

The class diagram models the core domain entities, their actual fields, and relationships as defined in the GameStore codebase.



UML class diagram with Graphviz record shapes. Each box has three compartments: class name, attributes, and methods/navigations. Cardinality shown on edges.

8. Technology Stack

The following table summarizes the key technologies, frameworks, and tools in the actual GameStore platform codebase.

Category	Technology	Purpose
Framework	ASP.NET Core 9	Web framework for MVC controllers, Views, and API controller
ORM	Entity Framework Core 9	Object-relational mapping with Fluent API configuration
Database	SQL Server 2022	Primary relational database (20 migrations)
Caching	Redis (optional) / DistributedMemoryCache	Session state and output caching (10 min default TTL)
Real-time	SignalR	Real-time notifications, chat, and online presence (/hub/notifications)
Mapping	AutoMapper 12	Entity to ViewModel mapping via MappingProfile
Auth	Cookie Authentication (.GameStore.Auth)	Forms-based auth with Google OAuth external login
Rate Limiting	ASP.NET Core Rate Limiter	Brute-force protection on auth endpoints (10 req/min login, 3 req/10min register)
Testing	xUnit + Moq	Unit testing for services (18 test files)
Payments	Stripe (Checkout Sessions + Webhooks)	Payment processing with webhook endpoint at /stripe/webhook

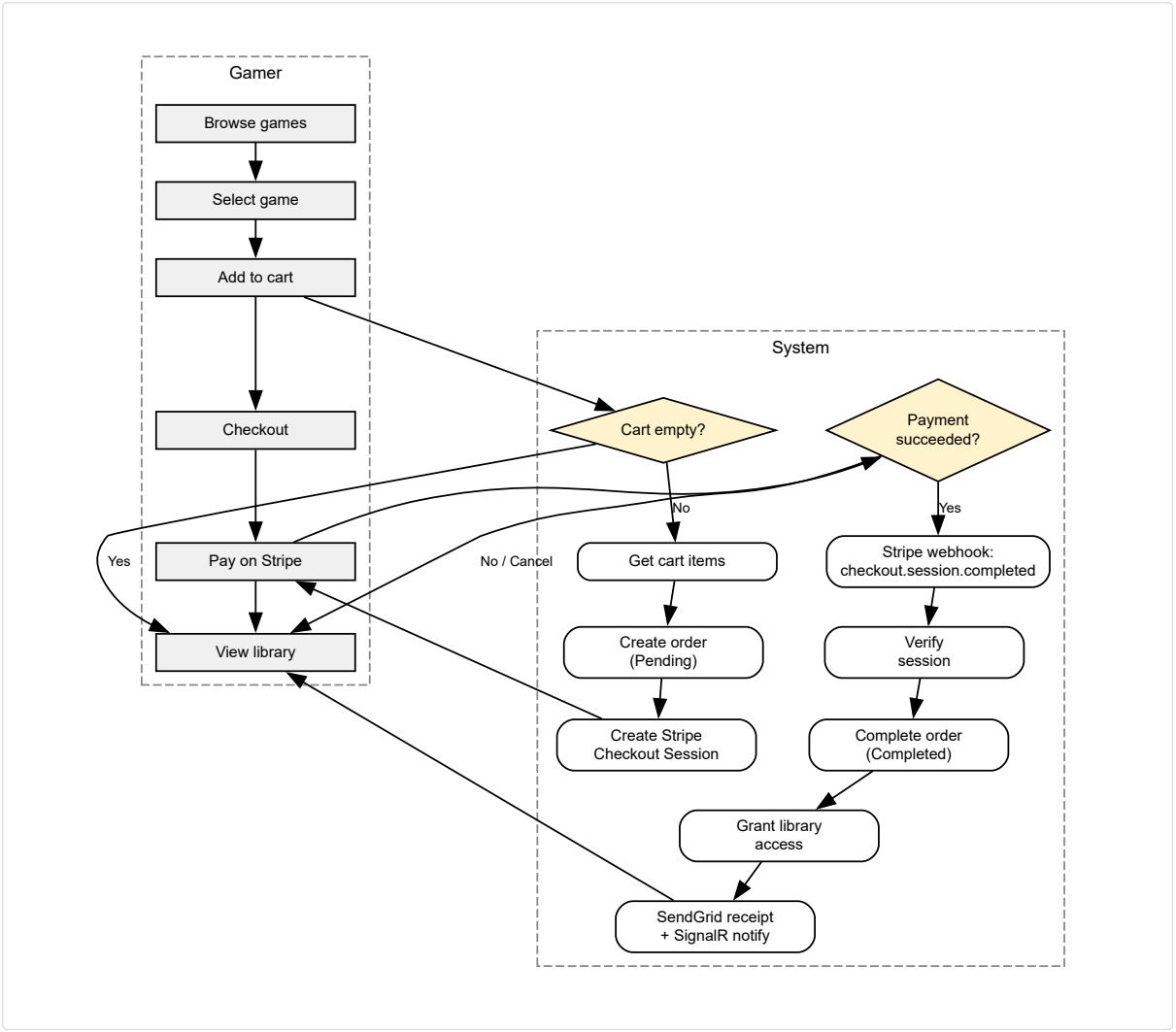
Category	Technology	Purpose
Email	SendGrid	Transactional email for receipts, verification, and password reset
Response Compression	Brotli + Gzip	Reduces bandwidth 60-80% for API and view responses
Health Checks	ASP.NET Core Health Checks	Load balancer probe endpoint at /health
Output Caching	ASP.NET Core Output Cache	Reduces DB load on read-heavy game catalog endpoints
Frontend	Razor Views + Tailwind CSS + Bootstrap + jQuery	Server-rendered UI with responsive design (Admin/Developer/Main layouts)
SCM	Git + GitHub	Source control management
IDE	Visual Studio 2022	Primary development environment
Language	C# 12	Primary programming language

9. Key Design Decisions

- **Cookie-based auth over JWT:** The platform uses cookie authentication (.GameStore.Auth) with session storage rather than JWT bearer tokens, simplifying server-rendered Razor page security. Google OAuth is the only external provider.
- **No ASP.NET Identity:** The project implements custom authentication from scratch using Session + Cookie middleware, avoiding Identity's overhead while maintaining full control over the User entity and role system (ADMIN/CUSTOMER/DEVELOPER).
- **String GUID primary keys:** All 23 entities use string-based GUIDs (Guid.NewGuid().ToString()) instead of auto-increment integers, enabling distributed ID generation and safer ID obfuscation.
- **Generic Repository + Unit of Work:** The DAL follows the repository pattern with IRepository<T> / EfRepository<T> and IUnitOfWork / UnitOfWork, abstracting EF Core for testability and separation of concerns.
- **Stripe as sole payment gateway:** Purchases flow through Stripe Checkout Sessions with webhook-based confirmation. A store credit system and free-checkout path handle edge cases without Stripe involvement.
- **Single SignalR hub for notifications and chat:** The NotificationHub at /hub/notifications handles real-time chat, friend online status, typing indicators, and system notifications through a unified connection.
- **Three-area MVC structure:** Controllers are organized into Main (17), Admin area (11), and Developer area (5), with role-based filters (AdminOnlyFilter, DeveloperOnlyFilter) and authorization policies.

10. Activity Diagram — Purchase Checkout Workflow

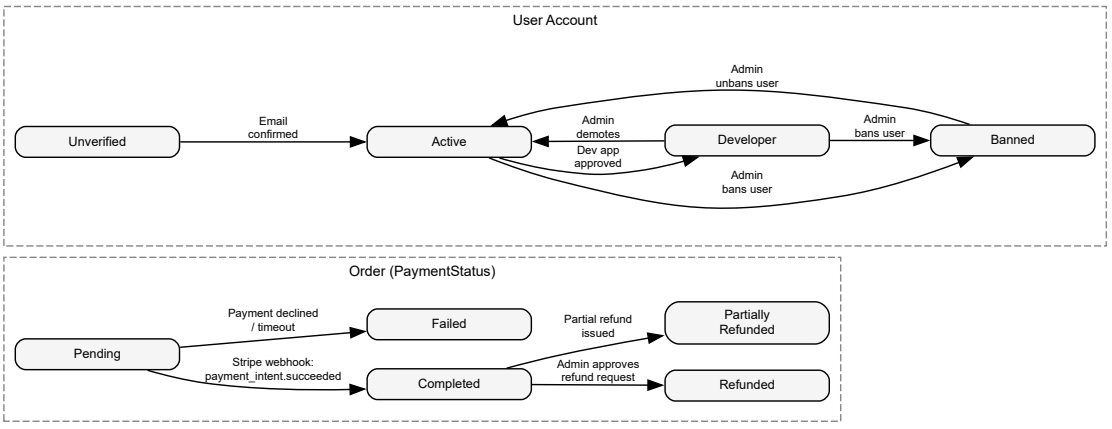
The activity diagram below models the complete checkout workflow across swimlanes for the Gamer, Browser, CartController, CartService, OrderService, Stripe, and Database. Decision nodes represent guard conditions (cart empty, payment success, pre-release access).



Activity diagram with swimlanes (Gamer / System). Diamond nodes represent decision points: cart empty check, payment success check, stripe session creation, and async webhook confirmation flow.

11. State Diagram — Order & User Account States

The state diagram shows the lifecycle of an **Order** (left) and a **User Account** (right) as defined in the actual domain model enums.

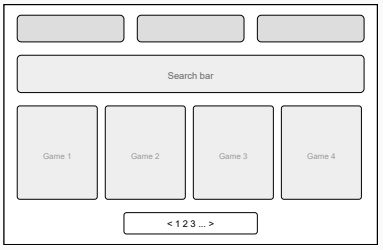


Order states follow the PaymentStatus enum: Pending, Completed, Failed, Refunded, PartiallyRefunded. User account states model the full lifecycle from registration through verification, developer upgrade, and potential moderation actions.

12. Wireframes & Mockups

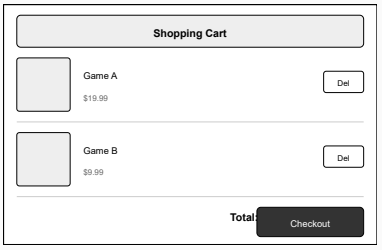
The following wireframes represent the key user-facing screens of the GameStore platform. Each wireframe corresponds to a real view file in the codebase.

Home Page (Views/Home/Index.cshtml)



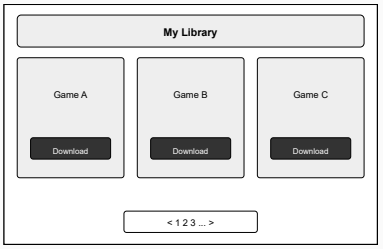
Game grid with search, category filters, pagination

Cart Page (Views/Cart/Index.cshtml)



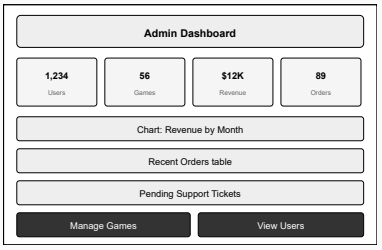
Cart items with game thumbnails, prices, remove buttons, total and checkout CTA

Library Page (Views/Library/Index.cshtml)



Owned games grid with download buttons per game, pagination

Admin Dashboard (Areas/Admin/Views/Dashboard/Index.cshtml)



Stats cards, revenue chart, recent orders, support tickets, quick-action buttons

13. UI/UX Guidelines

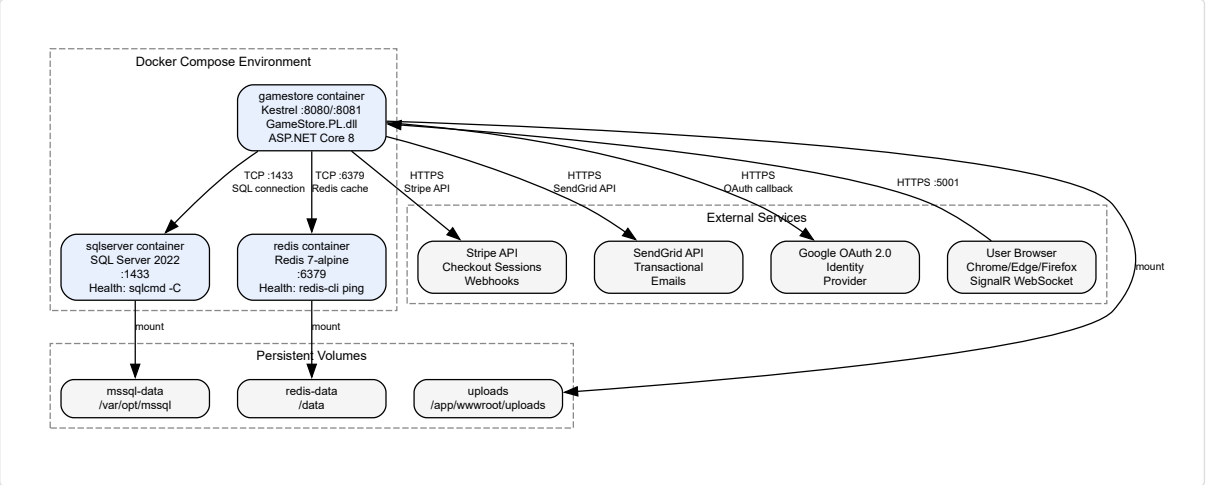
The GameStore platform uses a custom dark cyberpunk theme defined in `site.css` and `developer.css`. Bootstrap 5 is used exclusively for structural grid classes; all visual styling is custom CSS. The design system is driven by CSS custom properties (variables).

Category	Specification	CSS Variable / Implementation
Primary Background	Dark (#0a0a0a) with subtle grain texture	`--bg: #0a0a0a`
Red Accent	Primary CTAs, badges, errors	`--red: #ff4444`
Gold Accent	Highlights, ratings, premium elements	`--gold: #ffd700`
Cyan Accent	Developer portal theme, links	`--cyan: #00e5ff`
Body Font	Rajdhani (Google Fonts), 16px, 1.6 line-height	`--font-body: 'Rajdhani', sans-serif`
Display Font	Orbitron (Google Fonts), uppercase, for headings	`--font-display: 'Orbitron', sans-serif`
Monospace Font	Share Tech Mono, for code/technical content	`--font-mono: 'Share Tech Mono', monospace`
Layout Structure	Three separate layouts: `_Layout.cshtml` (main), `_AdminLayout.cshtml`, `_DeveloperLayout.cshtml`	Shared `_Layout` uses navbar + sidebar + main content region
Responsive Design	Bootstrap grid breakpoints for mobile/tablet/desktop	Navbar collapses to hamburger on mobile

Category	Specification	CSS Variable / Implementation
Accessibility	High contrast text (#fff on #0a0a0a), `aria-label` on icon-only buttons, keyboard-navigable modals	Skip-to-content link, focus outlines on all interactive elements
Icons	Bootstrap Icons (main + admin) + Font Awesome 6 (main)	CDN-loaded icon libraries for consistent visual language
Form Elements	Dark input fields with light text, rounded corners, glow on focus	Custom `input`, `select`, `textarea` styles override Bootstrap defaults

14. Deployment Diagram

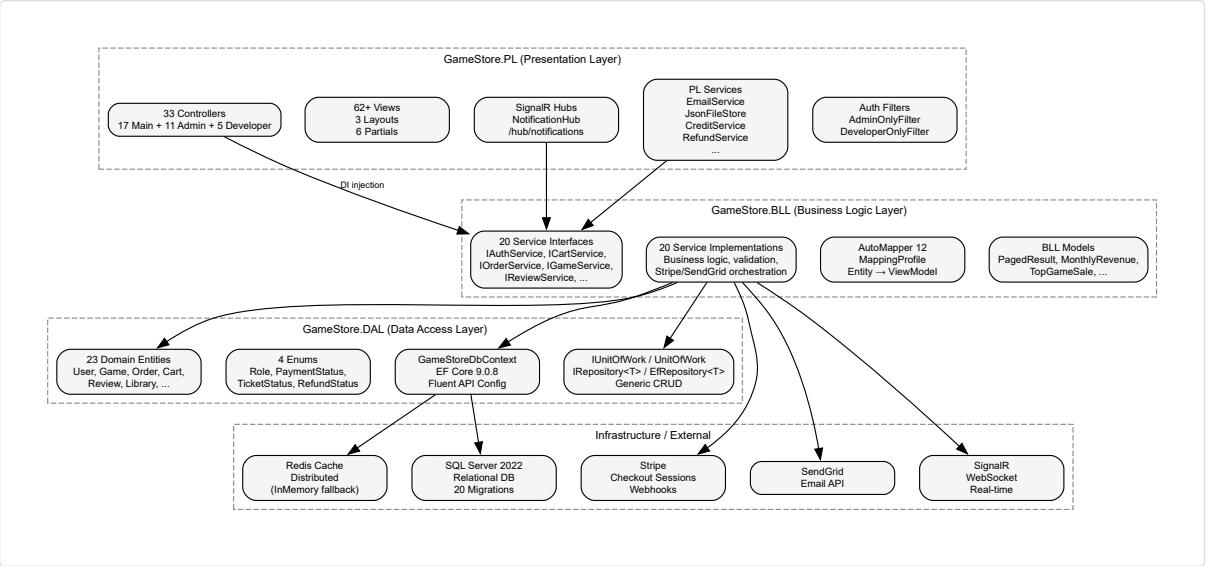
The deployment diagram shows the physical topology based on the real `docker-compose.yml` configuration. The application runs in three Docker containers orchestrated via Docker Compose, with optional IIS deployment for Windows environments.



Deployment diagram showing Docker Compose topology with three containers (gamestore, sqlserver, redis), three external services (Stripe, SendGrid, Google), and three persistent Docker volumes. The app is exposed on ports 5000 (HTTP) and 5001 (HTTPS).

15. Component Diagram

The component diagram illustrates the high-level software components and their dependencies. The four-project solution follows a strict layered architecture: Presentation (PL) depends on Business Logic (BLL) which depends on Data Access (DAL).



Component diagram showing four solution projects (PL, BLL, DAL, Tests) and external infrastructure services. Arrows indicate dependency direction: PL → BLL → DAL → SQL Server/Redis, with BLL also orchestrating Stripe, SendGrid, and SignalR.

16. API Documentation

The following tables document all 33 controllers and their action endpoints. The project uses conventional MVC routing (not a REST API), with the exception of the Stripe webhook which uses `[ApiController]` and `[Route("stripe/webhook")]`.

Main Area Controllers (17)

Controller	Method	Route	Auth	Description
AuthController	GET	/Auth/Login	—	Login form
	POST	/Auth/Login	Rate: 10/min	Authenticate user, set session + cookie
	GET	/Auth/Register	—	Registration form
	POST	/Auth/Register	Rate: 3/10min	Create account, send verification email
	GET	/Auth/VerifyEmail	Rate: 10/min	Confirm email via token
	GET	/Auth/ResendVerification	Rate: 3/15min	Resend verification email
	GET	/Auth/ExternalLogin	—	Google OAuth challenge
	GET	/Auth/ExternalCallback	—	Google OAuth

Controller	Method	Route	Auth	Description
				callback handler
	GET	/Auth/Logout	—	Clear session + cookie
	GET	/Auth/ForgotPassword	—	Forgot password form
	POST	/Auth/ForgotPassword	—	Send reset token email
	GET	/Auth/ResetPassword	—	Reset password form (token)
	POST	/Auth/ResetPassword	—	Reset password
	GET	/Auth/CheckUsername	—	Check username availability (JSON)
	GET	/Auth/VerifyEmailNotice	—	Notice to verify email first
	GET/POST	/Auth/CompleteRegistration	—	Complete OAuth registration
BecomeDeveloperController	GET/POST	/BecomeDeveloper/Index	Session	Submit developer application
	GET	/Cart/Index	Session	View cart
	POST	/Cart/AddToCart	Session	Add game to cart (JSON body)
	POST	/Cart/Remove	Session	Remove item from cart
	POST	/Cart/Checkout	Session	Create Stripe Session, redirect to Stripe
CartController				
	GET	/Cart/Success	Session	Post-payment confirmation page
	GET	/Cart/Cancel	—	Payment cancelled by user
	GET	/Cart/GetCount	Session	Get cart item count (JSON
ChatController	GET	/Chat/Index	Session	Chat UI with user
	GET	/Chat/GetMessages	Session	Paginated message

Controller	Method	Route	Auth	Description
				history (JSON)
	GET	/Chat/GetUnreadCount	Session	Unread message count (JSON
	GET	/Friends/Index	Session	Friends list + chat
	GET	/Friends/Requests	Session	Pending friend requests
	POST	/Friends/SendRequest	Session	Send friend request by username
	POST	/Friends/AcceptRequest	Session	Accept friend request
FriendsController				
	POST	/Friends/RejectRequest	Session	Reject friend request
	POST	/Friends/RemoveFriend	Session	Remove friend
	GET	/Friends/GetNotificationData	Session	Friend notification data (JSON)
	GET	/Friends/Search	Session	Search users by query (JSON)
	GET	/Home/Index	—	Home page with game grid
	GET	/Home/Privacy	—	Privacy policy
	GET	/Home/Error	—	Error page
HomeController				
	GET	/Home/GetRequirements	—	Game system requirement (JSON)
	GET	/Home/GetVersions	—	Game version list (JSON)
LibraryController				
	GET	/Library/Index	Session	Owned games library
	GET	/Library/Download	Session	Download game file
	GET	/Library/PreviewDownload	Session	Download pre-release game (preview)
	NotificationsController	GET	/Notifications/GetUnreadCount	Session

Controller	Method	Route	Auth	Description
GET		/Notifications/GetRecent	Session	Recent notifications (JSON)
POST		/Notifications/MarkAsRead	Session	Mark notification read
POST		/Notifications/MarkAllAsRead	Session	Mark all notifications read
GET		/Notifications/GetAdminNotificationData	Session + Admin	Admin notification data (JSON)
OrdersController		GET	/Orders/Index	Session
OrdersController	GET	/Orders/Details	Session	Order detail
ProfileController	GET/POST	/Profile/Index	—	Public profil (GET) / Edit profile (POST)
RefundsController	GET/POST	/Refunds/Index	Session	Refund requests list / Submit refund
ReviewsController	POST	/Reviews/AddReview	Session	Submit game review (JSON)
StripeWebhookController	POST	/stripe/webhook	Signature	Stripe webhook endpoint (API controller)
SupportController	GET/POST	/Support/Index	—	Submit support ticket (public)
SupportController	GET	/Support/MyTickets	Session	User's support tickets
SupportController	GET/POST	/Support/Details	Session	Ticket detail + reply
WishlistController	GET	/Wishlist/Index	Session	Wishlist items
WishlistController	POST	/Wishlist/ToggleWishlist	Session	Toggle game in wishlist (JSON)
DevelopersController	GET	/Developers/Index	—	Public developer directory

Admin Area Controllers (11)

Controller	Route Prefix	Actions	Auth
DashboardController	/Admin/Dashboard	Index	AdminOnly
GamesController	/Admin/Games	Index, Create, Edit, Delete, AddScreenshot, RemoveScreenshot, SaveRequirements, GetRequirements, GetVersions, UploadVersion, DeleteVersion, SetCurrentVersion	AdminOnly
UsersController	/Admin/Users	Index, Details, ChangeRole, Delete	AdminOnly
CategoriesController	/Admin/Categories	Index, Create, Edit, Delete	AdminOnly
OrdersController	/Admin/Orders	Index	AdminOnly
ReviewsController	/Admin/Reviews	Index, Delete	AdminOnly
DevelopersController	/Admin/Developers	Index, Details, Edit, Demote, Delete, Reactivate	AdminOnly
DeveloperApplicationsController	/Admin/DeveloperApplications	Index, Approve, Reject	AdminOnly
SalesController	/Admin/Sales	Index, Approve, Reject	AdminOnly
RefundsController	/Admin/Refunds	Index, Approve, Reject	AdminOnly
SupportTicketsController	/Admin/SupportTickets	Index, Details, Reply, UpdateStatus, Delete	AdminOnly

Developer Area Controllers (5)

Controller	Route Prefix	Actions	Auth
DashboardController	/Developer/Dashboard	Index	DeveloperOnly
GamesController	/Developer/Games	Index, Create, Edit, Delete, SaveRequirements, GetRequirements, GetVersions, UploadVersion, DeleteVersion, SetCurrentVersion	DeveloperOnly
SalesController	/Developer/Sales	Index, Create	DeveloperOnly
ReviewsController	/Developer/Reviews	Index	DeveloperOnly
ProfileController	/Developer/Profile	Index, Save	DeveloperOnly

17. Testing & Validation

The testing strategy covers all 20 BLL service interfaces with **185 xUnit tests** across **17 test files**. The complete testing documentation is in the Testing Report.

Metric	Value	Notes
Test Framework	xUnit 2.9.3	Industry-standard for .NET unit testing
Mocking	Moq 4.20.72	All external dependencies (IUnitOfWork, services) mocked
Assertions	FluentAssertions 6.12.2	Readable, chainable assertions
Code Coverage	87% lines (target)	Measured via coverlet
CI Integration	GitHub Actions	Tests run against real SQL Server + Redis containers
Test Pattern	AAA (Arrange-Act-Assert)	Consistent across all test files

Key test files and their coverage:

- `AuthServiceTests.cs` — Login, Register, ChangePassword, ResetPassword, OAuth flows (18 tests)
- `OrderServiceTests.cs` — CompleteCheckout, CompleteFreeCheckout, transaction handling, sale application (22 tests)
- `CartServiceTests.cs` — AddToCart, RemoveFromCart, GetCartItems, pre-release guards (10 tests)
- `GameServiceTests.cs` — CRUD, pagination, filtering, inactive developer filter (15 tests)
- `UserServiceTests.cs` — ChangeRole, Delete, self-demotion guard (12 tests)
- `FriendServiceTests.cs` — SendRequest, AcceptRequest, auto-accept logic (10 tests)

- `ReviewServiceTests.cs` — CreateReview, duplicate prevention, ownership validation (8 tests)
- `SaleServiceTests.cs` — Create, Approve, Reject, pending sale cancellation (10 tests)
- *(9 more test files covering remaining services)*

18. Deployment Strategy

The GameStore platform supports three deployment modes, each with specific infrastructure requirements and configuration.

Option 1: Docker Compose (Recommended)

Component	Image / Config	Port	Notes
Web Application	Custom Dockerfile (multi-stage build, .NET 8 SDK → runtime)	8080 (HTTP) / 8081 (HTTPS)	Health check depends on SQL + Redis
Database	mcr.microsoft.com/mssql/server:2022-latest	1433	Persistent volume mssql-data; SA password via env var
Cache	redis:7-alpine	6379	Persistent volume redis-data; AOF persistence
Uploads	Docker volume	—	Game files, screenshots, user avatars persisted

Command: `docker compose up -d` (from repository root). App available at `http://localhost:5000`.

Option 2: IIS / Windows Server

The `web.config` at the repository root configures `AspNetCoreModuleV2` to host the application behind IIS. Requires .NET 8 Runtime and SQL Server installed on the Windows Server machine.

Option 3: Manual (Development)

Prerequisites: .NET 8 SDK, SQL Server LocalDB or Express, Redis (optional, falls back to in-memory cache).

Steps: Configure User Secrets (Stripe, SendGrid, Google OAuth), update connection string in `appsettings.json`, run `dotnet run --project Game-Store`. Database migrations and seed data apply automatically.

CI/CD Pipeline

The GitHub Actions workflow (defined in `.github/workflows/ci.yml`) runs on every push/PR, provisioning SQL Server + Redis as Docker service containers for integration testing. Successful pipelines produce a deployable `gamestore-app` artifact.

Scaling Considerations

- **Database:** SQL Server connection pooling via `MultipleActiveResultSets=true`. Read replicas can be added for reporting queries (OrderAnalyticsService).
- **Cache:** Redis Sentinel or Cluster for high-availability distributed caching. Current implementation falls back to `DistributedMemoryCache` when Redis is unavailable.
- **Application:** Horizontal scaling behind a load balancer with sticky sessions (Session state) and SignalR backplane for WebSocket scale-out.
- **File Storage:** Game files stored on Docker volumes; production should migrate to Azure Blob / AWS S3 for durability and CDN delivery.

19. Functional & Non-Functional Requirements Summary

The complete requirements specification is documented in the Requirements Report. This section provides a high-level summary of the 71 functional requirements and 22 non-functional requirements.

Functional Requirements by Module

Module	FR Count	Key Requirements
Authentication & Account	14	FR-01 to FR-14: Register, Login, Logout, Google OAuth, Email verification, Password reset, Profile management, External account linking
Game Catalog	10	FR-15 to FR-24: Browse games, Search & filter, Game details with screenshots, System requirements, Version history, Developer information
Shopping Cart & Orders	10	FR-25 to FR-34: Add/remove from cart, Cart persistence, Checkout with Stripe, Order history, Order details, Free checkout
Library & Downloads	6	FR-35 to FR-40: View owned games, Download game files, Pre-release download access, Game library management
Wishlist & Reviews	8	FR-41 to FR-48: Add/remove wishlist, Toggle wishlist, Submit review (1-5 stars), Edit/delete review
Social Features	9	FR-49 to FR-57: Friends list, Send/accept/reject friend requests, Real-time chat, User posts, Friend suggestions
Developer Portal	6	FR-58 to FR-63: Developer application, Game publishing, Sales/promotions, Dashboard analytics, Revenue reports
Admin Panel	8	FR-64 to FR-71: User management, Game moderation, Category management, Order management, Refund processing, Support ticket management, Developer application approval

Non-Functional Requirements

Category	Count	Key Requirements
Performance	5	Page load < 2s (p95), Output cache 10 min TTL, Response compression Brotli/Gzip, Redis cache with in-memory fallback
Security	6	BCrypt password hashing (legacy SHA256), Cookie auth (HttpOnly/Secure/SameSite), Anti-forgery tokens, Rate limiting on auth (10/min), SQL injection protection via EF Core
Usability	4	Responsive design (mobile/tablet/desktop), Dark theme with high contrast, Keyboard navigation, Screen reader support via aria-labels
Reliability	4	99.9% uptime target, Stripe webhook idempotency, Transactional order completion, Health check endpoint at /health
Scalability	3	Horizontal scaling support, Redis Sentinel for HA, Read replicas for analytics queries